gemstone-py Type-Safe Smalltalk Codegen

Generating typed wrappers from Protocol classes

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

Type-Safe Smalltalk Codegen

`gemstone-codegen` turns registered Python `Protocol` classes into concrete `TypedOop` wrappers. The generated files are ordinary Python modules, so IDEs can autocomplete methods and your application code no longer has to spell the same Smalltalk strings at every call site.

Install

The generator ships with `gemstone-py`:

```
python -m pip install gemstone-py
gemstone-codegen --help
```

For source checkouts:

```
python -m pip install -e ".[examples,dev]"
python -m gemstone_py.codegen --help
```

Define Protocols

Create a module with one or more `@gemstone_class(...)` protocols:

```
from typing import Protocol
from gemstone_py import gemstone_class, gemstone_selector

@gemstone_class("OkzBooking", async_=True)
class OkzBookingProto(Protocol):
    status: str

    @classmethod
    @gemstone_selector("findById:")
    def find_by_id(cls, booking_id: str) -> "OkzBookingProto":
        ...

    def mark_paid(self, at_posix_seconds: int) -> None:
        ...

    @gemstone_selector("transferTo:byUserId:")
    def transfer(self, user_id: int, by_user_id: int) -> None:
        ...
```

`async_=True` asks the generator to emit both sync and async wrappers.

Generate Wrappers

Run the generator from the repository or application root:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated
```

The output is meant to be checked in. That keeps reviews, editor indexing, and type checking predictable:

```
git add examples/typed_access/codegen_demo/generated
```

Use `--check` in CI or pre-commit hooks to catch drift:

```
gemstone-codegen \
  --module examples.typed_access.codegen_demo.models \
  --output examples/typed_access/codegen_demo/generated \
  --check
```

Selector Mapping

Default mapping is intentionally small:

Python shape	Generated Smalltalk selector
`status`	`status`
`find_by_id(id)`	`findById:`
`mark_paid(at)`	`markPaid:`
`transfer_to(user_id, by_user_id)`	`transferTo:byUserId:`

For selectors that do not map cleanly, use `@gemstone_selector(...)`. The explicit selector must have the same keyword count as the Python method has arguments.

Sync Usage

Generated class-side methods take a session and build the Smalltalk source:

```
from gemstone_py import GemStoneConfig, GemStoneSession
from examples.typed_access.codegen_demo.generated import OkzBooking

with GemStoneSession(config=GemStoneConfig.from_env()) as session:
    booking = OkzBooking.find_by_id(session, "B-1001")
    print(booking.status)
    booking.mark_paid(1_779_912_000)
```

That class-side call evaluates:

```
OkzBooking findById: 'B-1001'
```

Instance methods call `perform_value(...)` through the session associated with the returned `TypedOop`.

Async Usage

When the Protocol is registered with `async_=True`, the generator also emits an `Async...` wrapper:

```
from fastapi import Depends, FastAPI
from gemstone_py import GemStoneConfig
from gemstone_py.aio import AsyncSession
from gemstone_py.aio.fastapi import session_dependency
from examples.typed_access.codegen_demo.generated import AsyncOkzBooking

app = FastAPI()
get_gemstone =
session_dependency(config=GemStoneConfig.from_env(require_credentials=False))

@app.get("/bookings/{booking_id}")
async def booking_status(
    booking_id: str,
    session: AsyncSession = Depends(get_gemstone),
) -> dict[str, str]:
    booking = await AsyncOkzBooking.find_by_id(session, booking_id)
    return {"status": str(await booking.status())}
```

Current Scope

The first generator pass handles:

- annotated fields and `@property` methods as no-argument sends
- instance methods as `perform_value(...)` sends
- class methods as class-side Smalltalk source strings

- string, integer, float, boolean, and `None` literals in class-side calls
- explicit selector overrides with `@gemstone_selector(...)`
- checked-in sync and async generated modules

It does not infer return OOP types, marshal arbitrary Python objects into class-side source literals, or replace hand-written Smalltalk for complex queries. Use it for method-shaped object access and keep raw `session.eval(...)` or `SmalltalkBridge` calls where a full Smalltalk expression is clearer.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.